



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Design Patterns (cont.)

Instructor: Le Thi Ngoc Hanh, Ph.D

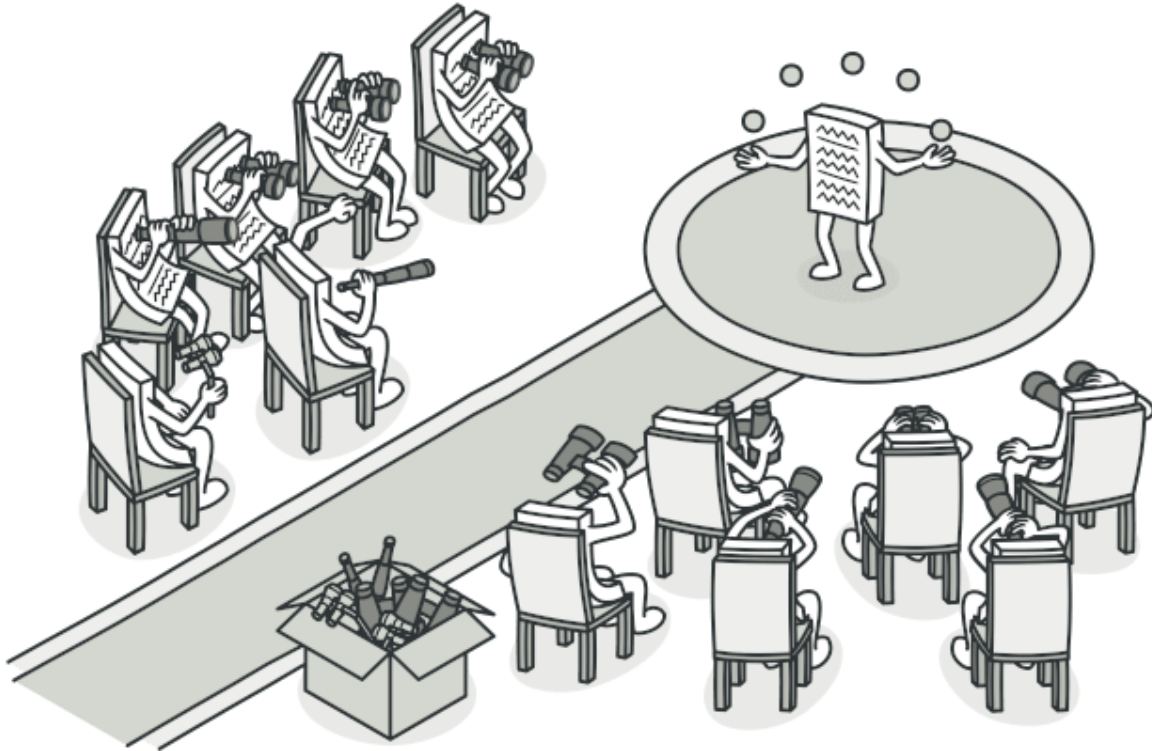
ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

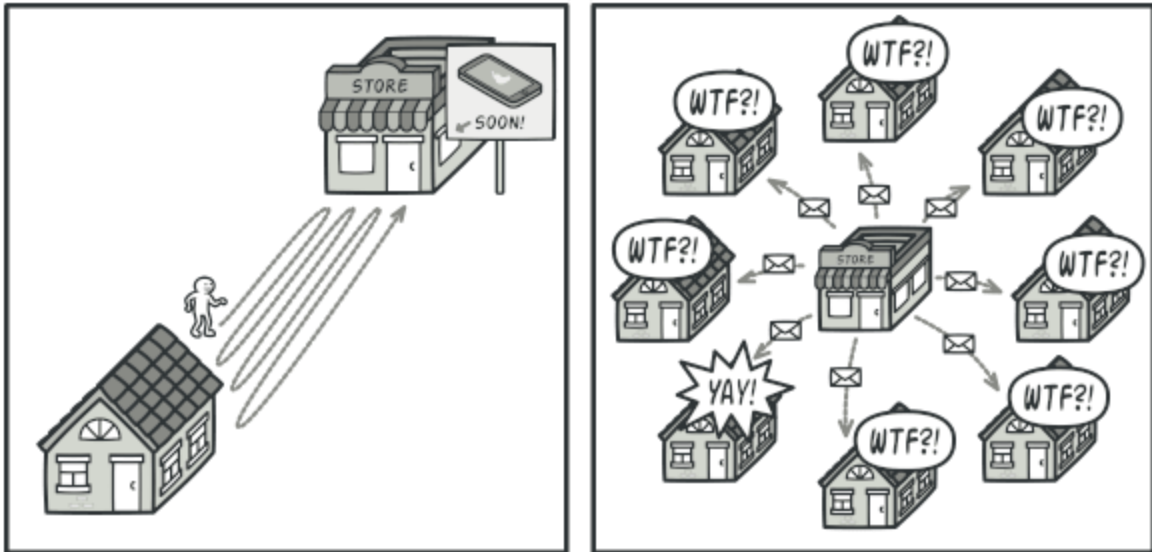
Lecture Objectives

- ☑ Understand the Concept of Design Patterns
- ☑ Recognize the significance of Design Patterns in Software Development
- ☑ Apply Design Patterns in real world case study

Observer Pattern



Observer Pattern



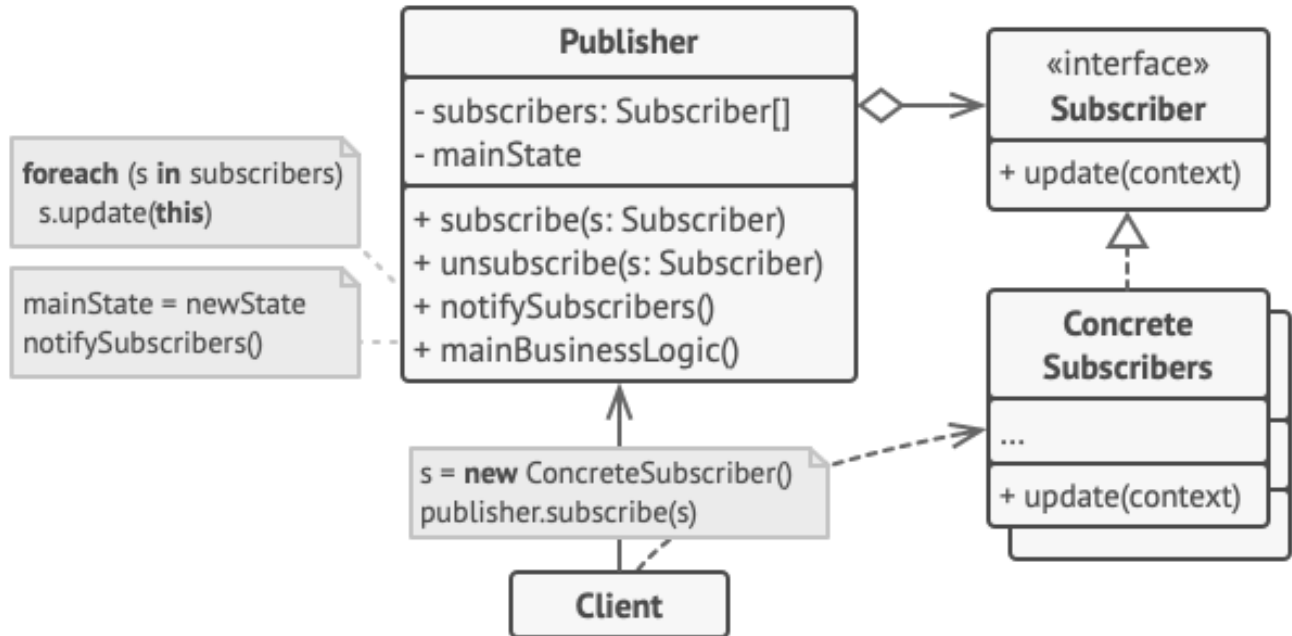
Observer Pattern

Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.

Components:

- **Subject:** Maintains a list of Observers and notifies them of state changes.
- **Observer:** Defines an interface for objects that should be notified of changes.
- **ConcreteSubject:** Implements the Subject interface and stores state.
- **ConcreteObserver:** Implements the Observer interface and updates its state based on notifications.

Observer Pattern



Sample code

```
// Observer interface  
interface Subscriber {  
    void update(Publisher publisher);  
}
```

```
// Subject / Publisher
class Publisher {
    private List<Subscriber> subscribers = new ArrayList<>();
    private String mainState;
    public void subscribe(Subscriber s) {
        subscribers.add(s);
    }
    public void unsubscribe(Subscriber s) {
        subscribers.remove(s);
    }
    public void notifySubscribers() {
        for (Subscriber s : subscribers) {
            s.update(this);
        }
    }
    public void mainBusinessLogic(String newState) {
        System.out.println("Publisher: changing mainState...");
        this.mainState = newState;

        notifySubscribers();
    }
    public String getMainState() {
        return mainState;
    }
}
```

```
// Concrete Observer
class ConcreteSubscriber implements Subscriber {
    private String name;

    public ConcreteSubscriber(String name) {
        this.name = name;
    }

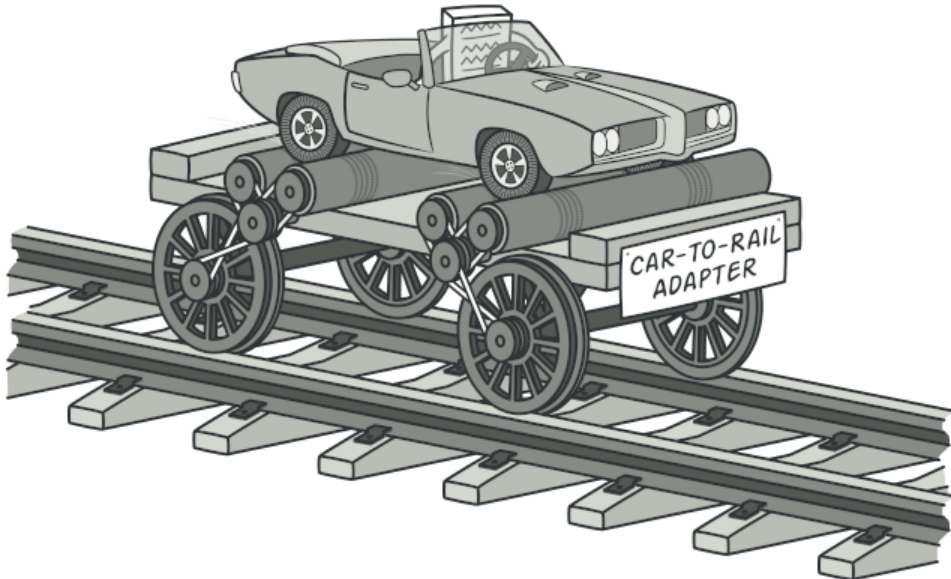
    @Override
    public void update(Publisher publisher) {
        System.out.println(
            name + " received update. New state = " + publisher.getMainState()
        );
    }
}
```

```
// Client code
```

```
public class Main {  
    public static void main(String[] args) {  
        Publisher publisher = new Publisher();  
  
        Subscriber s1 = new ConcreteSubscriber("Subscriber A");  
        Subscriber s2 = new ConcreteSubscriber("Subscriber B");  
        Subscriber s3 = new ConcreteSubscriber("Subscriber C");  
  
        publisher.subscribe(s1);  
        publisher.subscribe(s2);  
        publisher.subscribe(s3);  
  
        publisher.mainBusinessLogic("State 1");  
  
        System.out.println("--- Unsubscribe Subscriber B ---");  
        publisher.unsubscribe(s2);  
  
        publisher.mainBusinessLogic("State 2");  
    }  
}
```

Adapter Pattern

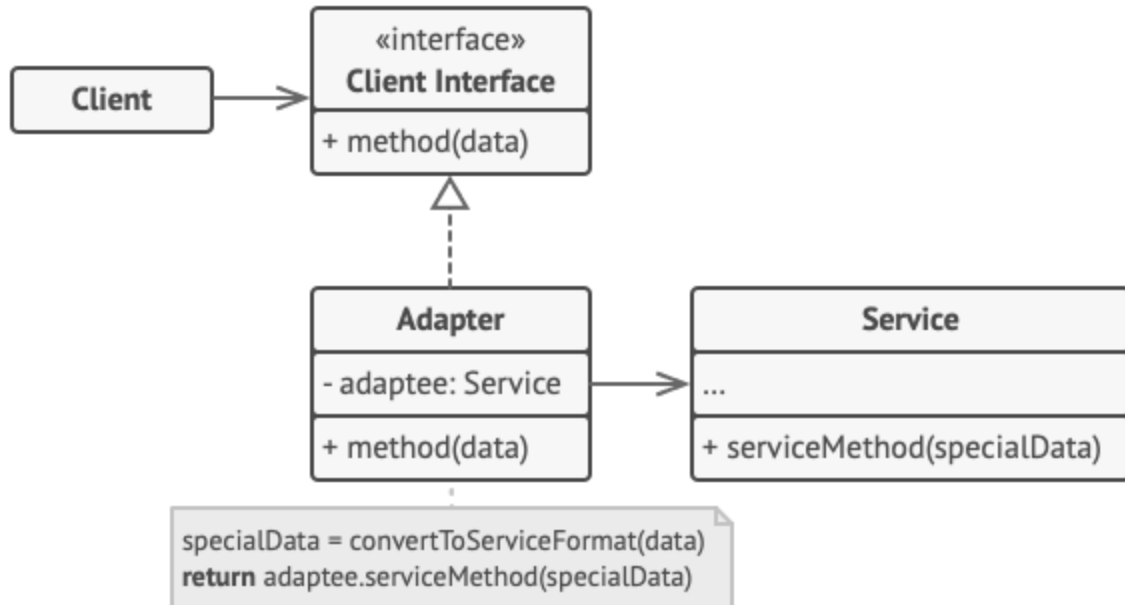
Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.



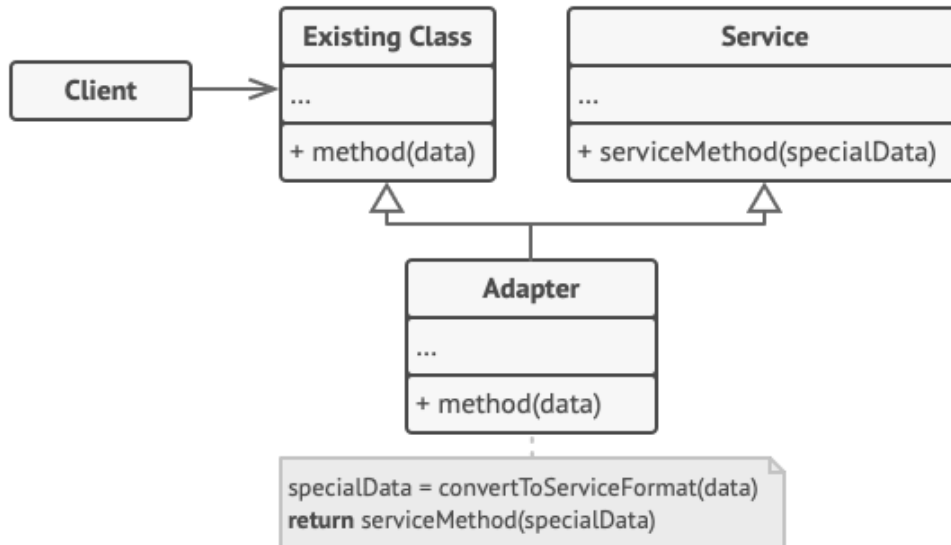
Adapter Pattern

- ✧ **Adapter** is a structural design pattern that allows objects with incompatible interfaces to work together by converting the interface of one class into another interface expected by the client.
- ✧ **Components:**
 - **Target:** Defines the interface that the client expects to use.
 - **Adapter:** Implements the Target interface and translates client requests to the Adaptee.
 - **Adaptee:** Existing class with useful functionality, but its interface is incompatible with the client's expected interface.
 - **Client:** Uses the Target interface and does not know whether it is working with an original class or an adapted class.

Object adapter: This implementation uses the object composition principle: the adapter implements the interface of one object and wraps the other one. It can be implemented in all popular programming languages.



Class adapter: This implementation uses inheritance: the adapter inherits interfaces from both objects at the same time. Note that this approach can only be implemented in programming languages that support multiple inheritance, such as C++.



Sample Code

```
// Target interface: the interface expected by the Client
interface ClientInterface {
    String method(String data);
}
```

```
// Existing service class with an incompatible interface
class Service {
    public String serviceMethod(String specialData) {
        return "Service is processing: " + specialData;
    }
}
```

Sample Code

```
// Adapter class
class Adapter implements ClientInterface {
    private Service adaptee;

    public Adapter(Service adaptee) {
        this.adaptee = adaptee;
    }

    @Override
    public String method(String data) {
        String specialData = convertToServiceFormat(data);
        return adaptee.serviceMethod(specialData);
    }

    private String convertToServiceFormat(String data) {
        return "[Converted Data] " + data;
    }
}
```

Sample Code

```
// Client code
public class Main {
    public static void main(String[] args) {
        Service service = new Service();

        ClientInterface adapter = new Adapter(service);

        String result = adapter.method("normal client data");

        System.out.println(result);
    }
}
```